

Spring 2026
CS 424/524: Intelligent Mobile Robotics
Assignment 2

GOAL

Navigation: Students will learn how to use ROS packages for SLAM, localization, and navigation, and how to create programs to drive a physical robot to visit specific positions. The first goal of this assignment is to give students first-hand experience of working on real robots.

Vision: Students will learn to write code in ROS that is based on visual input (RGB-D) to control robot platforms.

Students can either work individually or form 2-student teams.

INSTRUCTIONS (2 parts)

PART 1.

Part 1 can be done using either C++ or Python (no preference). Students will first use the ROS package, GMapping, to build a map that covers three locations (L1, L2, L3) that are at least 25 feet (about 8 meters or 10 steps) away from each other. Each team can define their own three locations.

The next step is autonomous navigation: let the robot start from L1, visit L2, visit L3, and then return to L1. After the robot starts to move, there should be no human involvement in any form.

1. Turn on the robot (both computer and robot base).
2. Open a terminal and run the following command to start the robot base driver. The “screen” part is optional and enables more printouts that can be useful for debugging.

```
roslaunch turtlebot_bringup minimal.launch --screen
```

3. Open another terminal and run the following command (for gmapping, which is a ROS implementation of FastSLAM).

```
roslaunch turtlebot_navigation gmapping_demo.launch --screen
```

4. Open another terminal and run the following to start ‘rviz’ for visualization:

```
roslaunch turtlebot_rviz_launchers view_navigation.launch --screen
```

5. Finally, in a new terminal, run the following command for teleoperation.

```
roslaunch turtlebot_teleop keyboard_teleop.launch --screen
```

6. Keep the teleop terminal being focused, use the keys of “i, j, m, l” to control the robot moving forward, turning left, moving backward, and turning right.

Steps 2-6 are demonstrated in this video (ssh is unnecessary):

<https://youtu.be/9efBzYAi1QI>

7. After driving the robot around, a map of the lab is expected to be generated and shown in rviz. Run the following command to save the map:

```
roslaunch map_server map_saver -f /path/to/your_map
```

8. Now you have a map (that includes a yaml file and a pgm file) ready to be used for autonomous navigation.

To do that, simply close all your ROS applications, then in a new terminal run the command in Step 2 to start the robot base driver, and run the following command for navigation:

```
roslaunch turtlebot_navigation amcl_demo.launch map_file:=/path/to/your_map.yaml
```

To visualize the map and robot's pose, the command in Step 4 needs to be run as well. This video shows the steps of autonomous navigation (ssh is unnecessary):

<https://youtu.be/iGZPWjddans>

9. The instructions above can be used for SLAM and autonomous navigation WITHOUT creating your own code.

In order to let the robot visit the locations, students will need to write code to send navigation goals to the robot (one at a time). More specifically, goal messages (in the ROS type of `geometry_msgs/PoseStamped`) need to be sent to the ROS topic of `/move_base_simple/goal`.

PART 2.

Students will create code to enable the robot to follow a ball in red (or any specific) color while keeping a distance of 3 feet (about 1 meter) from the ball.

1. The following command will start the driver of Turtlebot base:

```
roslaunch turtlebot_bringup minimal.launch --screen
```

2. The following command activates the rgb camera. After running this command, a window will pop up showing the rgb frames.

```
roslaunch usb_cam usb_cam-test.launch -screen
```

3. It happens that the laptop's camera is used (instead of the Astra camera). In this case, the following parameter needs to be changed to `video0` or `video1`, whichever works, in the launch file of `usb_cam-test.launch`

```
<param name="video_device" value="/dev/video1" />
```

4. The following command starts the depth camera:

```
roslaunch astra_launch astra.launch --screen
```

5. You can use the tool of `image_view` to see if the rgb and/or depth cameras work or not.

```
roslaunch image_view image_view image:=/topic/of/images
```

6. Finally, students will create code to, based on the rgb and depth frames, drive the robot platform to follow a ball of red color while maintaining a distance of one meter.
7. To help students get used to processing rgb and depth frames, the instructor created a ROS package that finds the “most red” pixel in RGB frames while drawing a circle around it. The code is available on Github:

https://raw.githubusercontent.com/shiqizhang6/robotics_teaching/refs/heads/master/assign3/src/code.cpp

8. The following video shows the expected behaviors of your robot (While the code associated to the video is available, please be cautious that the code is very outdated and starting from there may require more effort than developing your own code directly):

<https://youtu.be/FO-vBwJLUiY>

9. Bonus points (of up to 20%) will be given, if the student’ solution does not require the RGB stream of the visual sensor. This can be easily tested by covering the RGB camera, turning off the light, or changing the ball color.

Please let the instructor and the TA know, if your code does not require the RGB sensor readings.

WHAT TO TURN IN

Create a new repository on [this Github page](#), and name it in the following fashion (This should match with the package that you created in this assignment.)

For example, the file name should be “p2_zhangs” for the instructor or “p2_zhangs_aoloo-r” for the team of the instructor and the TA. Make sure that extracting this file generates a folder of the same name that contains the following subfolders

“launch”, where your launch file locates

“src”, where stores your C++ codes

“script”, where stores your Python codes

“misc”, where you store your “message” text, and the image of your name initials.

Please include a readme file in this subfolder to list your collaborator’s name (if any).

and the following files as necessary:

CMakeLists.txt
package.xml

Your program will be run in two steps:

1. Git-pull the repository under the path of 'catkin_ws/src'.
2. The following two commands will be run under an arbitrary directory, where the package name will be changed based on student names. The two launch files correspond to the two parts of this assignment.

```
roslaunch p2_zhangs p2a.launch
```

```
roslaunch p2_zhangs p2b.launch
```

After pushing your changes to your remote repository, submit `submission.txt` to the P2 assignment on BrightSpace. You can create the file using the commands below.

None

```
[cs424cs542-ta@aoloo ~/p2-xxx]$ touch submission.txt
```

```
[cs424cs524-ta@aoloo ~/p2-xxx]$ echo  
"https://github.com/https://github.com/bu-cs-424-524/$(basename -s  
.git $(git config --get remote.origin.url))/tree/$(git log  
--grep="$keyword" -n 1 --format="%H")" > submission.txt
```

Students will submit a link to a video in the cloud (Google Drive, Dropbox, etc) with the “anyone with the link can view” permission. The video shows the robot autonomously navigating the locations as required in PART 1.

Upload your submission to Brightspace.

The instructor reserves the right to request a live demonstration when needed. In such cases, please do not take it personally.